

previous definitions were completely subjective, and hence limited the extent to which we could discuss certain issues in concrete terms.

In particular, the first of our definitions, which was tied to the specific implementation of an algorithm, turned efficiency into a moving target: as processor speeds increase, more and more algorithms fall under this notion of efficiency. Our definition in terms of polynomial time is much more an absolute notion; it is closely connected with the idea that each problem has an intrinsic level of computational tractability: some admit efficient solutions, and others do not.

2.2 Asymptotic Order of Growth

Our discussion of computational tractability has turned out to be intrinsically based on our ability to express the notion that an algorithm's worst-case running time on inputs of size n grows at a rate that is at most proportional to some function $f(n)$. The function $f(n)$ then becomes a bound on the running time of the algorithm. We now discuss a framework for talking about this concept.

We will mainly express algorithms in the pseudo-code style that we used for the Gale-Shapley algorithm. At times we will need to become more formal, but this style of specifying algorithms will be completely adequate for most purposes. When we provide a bound on the running time of an algorithm, we will generally be counting the number of such pseudo-code steps that are executed; in this context, one *step* will consist of assigning a value to a variable, looking up an entry in an array, following a pointer, or performing an arithmetic operation on a fixed-size integer.

When we seek to say something about the running time of an algorithm on inputs of size n , one thing we could aim for would be a very concrete statement such as, "On any input of size n , the algorithm runs for at most $1.62n^2 + 3.5n + 8$ steps." This may be an interesting statement in some contexts, but as a general goal there are several things wrong with it. First, getting such a precise bound may be an exhausting activity, and more detail than we wanted anyway. Second, because our ultimate goal is to identify broad classes of algorithms that have similar behavior, we'd actually like to classify running times at a coarser level of granularity so that similarities among different algorithms, and among different problems, show up more clearly. And finally, extremely detailed statements about the number of steps an algorithm executes are often—in a strong sense—meaningless. As just discussed, we will generally be counting steps in a pseudo-code specification of an algorithm that resembles a high-level programming language. Each one of these steps will typically unfold into some fixed number of primitive steps when the program is compiled into

an intermediate representation, and then into some further number of steps depending on the particular architecture being used to do the computing. So the most we can safely say is that as we look at different levels of computational abstraction, the notion of a “step” may grow or shrink by a constant factor—for example, if it takes 25 low-level machine instructions to perform one operation in our high-level language, then our algorithm that took at most $1.62n^2 + 3.5n + 8$ steps can also be viewed as taking $40.5n^2 + 87.5n + 200$ steps when we analyze it at a level that is closer to the actual hardware.

O , Ω , and Θ

For all these reasons, we want to express the growth rate of running times and other functions in a way that is insensitive to constant factors and low-order terms. In other words, we’d like to be able to take a running time like the one we discussed above, $1.62n^2 + 3.5n + 8$, and say that it grows like n^2 , up to constant factors. We now discuss a precise way to do this.

Asymptotic Upper Bounds Let $T(n)$ be a function—say, the worst-case running time of a certain algorithm on an input of size n . (We will assume that all the functions we talk about here take nonnegative values.) Given another function $f(n)$, we say that $T(n)$ is $O(f(n))$ (read as “ $T(n)$ is order $f(n)$ ”) if, for sufficiently large n , the function $T(n)$ is bounded above by a constant multiple of $f(n)$. We will also sometimes write this as $T(n) = O(f(n))$. More precisely, $T(n)$ is $O(f(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ so that for all $n \geq n_0$, we have $T(n) \leq c \cdot f(n)$. In this case, we will say that T is *asymptotically upper-bounded by f* . It is important to note that this definition requires a constant c to exist that works for *all* n ; in particular, c cannot depend on n .

As an example of how this definition lets us express upper bounds on running times, consider an algorithm whose running time (as in the earlier discussion) has the form $T(n) = pn^2 + qn + r$ for positive constants p , q , and r . We’d like to claim that any such function is $O(n^2)$. To see why, we notice that for all $n \geq 1$, we have $qn \leq qn^2$, and $r \leq rn^2$. So we can write

$$T(n) = pn^2 + qn + r \leq pn^2 + qn^2 + rn^2 = (p + q + r)n^2$$

for all $n \geq 1$. This inequality is exactly what the definition of $O(\cdot)$ requires: $T(n) \leq cn^2$, where $c = p + q + r$.

Note that $O(\cdot)$ expresses only an upper bound, not the exact growth rate of the function. For example, just as we claimed that the function $T(n) = pn^2 + qn + r$ is $O(n^2)$, it’s also correct to say that it’s $O(n^3)$. Indeed, we just argued that $T(n) \leq (p + q + r)n^2$, and since we also have $n^2 \leq n^3$, we can conclude that $T(n) \leq (p + q + r)n^3$ as the definition of $O(n^3)$ requires. The fact that a function can have many upper bounds is not just a trick of the notation; it shows up in the analysis of running times as well. There are cases

where an algorithm has been proved to have running time $O(n^3)$; some years pass, people analyze the same algorithm more carefully, and they show that in fact its running time is $O(n^2)$. There was nothing wrong with the first result; it was a correct upper bound. It's simply that it wasn't the "tightest" possible running time.

Asymptotic Lower Bounds There is a complementary notation for lower bounds. Often when we analyze an algorithm—say we have just proven that its worst-case running time $T(n)$ is $O(n^2)$ —we want to show that this upper bound is the best one possible. To do this, we want to express the notion that for arbitrarily large input sizes n , the function $T(n)$ is *at least* a constant multiple of some specific function $f(n)$. (In this example, $f(n)$ happens to be n^2 .) Thus, we say that $T(n)$ is $\Omega(f(n))$ (also written $T(n) = \Omega(f(n))$) if there exist constants $\epsilon > 0$ and $n_0 \geq 0$ so that for all $n \geq n_0$, we have $T(n) \geq \epsilon \cdot f(n)$. By analogy with $O(\cdot)$ notation, we will refer to T in this case as being *asymptotically lower-bounded by* f . Again, note that the constant ϵ must be fixed, independent of n .

This definition works just like $O(\cdot)$, except that we are bounding the function $T(n)$ from below, rather than from above. For example, returning to the function $T(n) = pn^2 + qn + r$, where p , q , and r are positive constants, let's claim that $T(n) = \Omega(n^2)$. Whereas establishing the upper bound involved "inflating" the terms in $T(n)$ until it looked like a constant times n^2 , now we need to do the opposite: we need to reduce the size of $T(n)$ until it looks like a constant times n^2 . It is not hard to do this; for all $n \geq 0$, we have

$$T(n) = pn^2 + qn + r \geq pn^2,$$

which meets what is required by the definition of $\Omega(\cdot)$ with $\epsilon = p > 0$.

Just as we discussed the notion of "tighter" and "weaker" upper bounds, the same issue arises for lower bounds. For example, it is correct to say that our function $T(n) = pn^2 + qn + r$ is $\Omega(n)$, since $T(n) \geq pn^2 \geq pn$.

Asymptotically Tight Bounds If we can show that a running time $T(n)$ is both $O(f(n))$ and also $\Omega(f(n))$, then in a natural sense we've found the "right" bound: $T(n)$ grows exactly like $f(n)$ to within a constant factor. This, for example, is the conclusion we can draw from the fact that $T(n) = pn^2 + qn + r$ is both $O(n^2)$ and $\Omega(n^2)$.

There is a notation to express this: if a function $T(n)$ is both $O(f(n))$ and $\Omega(f(n))$, we say that $T(n)$ is $\Theta(f(n))$. In this case, we say that $f(n)$ is an *asymptotically tight bound* for $T(n)$. So, for example, our analysis above shows that $T(n) = pn^2 + qn + r$ is $\Theta(n^2)$.

Asymptotically tight bounds on worst-case running times are nice things to find, since they characterize the worst-case performance of an algorithm

precisely up to constant factors. And as the definition of $\Theta(\cdot)$ shows, one can obtain such bounds by closing the gap between an upper bound and a lower bound. For example, sometimes you will read a (slightly informally phrased) sentence such as “An upper bound of $O(n^3)$ has been shown on the worst-case running time of the algorithm, but there is no example known on which the algorithm runs for more than $\Omega(n^2)$ steps.” This is implicitly an invitation to search for an asymptotically tight bound on the algorithm’s worst-case running time.

Sometimes one can also obtain an asymptotically tight bound directly by computing a limit as n goes to infinity. Essentially, if the ratio of functions $f(n)$ and $g(n)$ converges to a positive constant as n goes to infinity, then $f(n) = \Theta(g(n))$.

(2.1) Let f and g be two functions that

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$$

exists and is equal to some number $c > 0$. Then $f(n) = \Theta(g(n))$.

Proof. We will use the fact that the limit exists and is positive to show that $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$, as required by the definition of $\Theta(\cdot)$.

Since

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c > 0,$$

it follows from the definition of a limit that there is some n_0 beyond which the ratio is always between $\frac{1}{2}c$ and $2c$. Thus, $f(n) \leq 2cg(n)$ for all $n \geq n_0$, which implies that $f(n) = O(g(n))$; and $f(n) \geq \frac{1}{2}cg(n)$ for all $n \geq n_0$, which implies that $f(n) = \Omega(g(n))$. ■

Properties of Asymptotic Growth Rates

Having seen the definitions of O , Ω , and Θ , it is useful to explore some of their basic properties.

Transitivity A first property is *transitivity*: if a function f is asymptotically upper-bounded by a function g , and if g in turn is asymptotically upper-bounded by a function h , then f is asymptotically upper-bounded by h . A similar property holds for lower bounds. We write this more precisely as follows.

(2.2)

(a) If $f = O(g)$ and $g = O(h)$, then $f = O(h)$.

(b) If $f = \Omega(g)$ and $g = \Omega(h)$, then $f = \Omega(h)$.

Proof. We'll prove part (a) of this claim; the proof of part (b) is very similar.

For (a), we're given that for some constants c and n_0 , we have $f(n) \leq cg(n)$ for all $n \geq n_0$. Also, for some (potentially different) constants c' and n'_0 , we have $g(n) \leq c'h(n)$ for all $n \geq n'_0$. So consider any number n that is at least as large as both n_0 and n'_0 . We have $f(n) \leq cg(n) \leq cc'h(n)$, and so $f(n) \leq cc'h(n)$ for all $n \geq \max(n_0, n'_0)$. This latter inequality is exactly what is required for showing that $f = O(h)$. ■

Combining parts (a) and (b) of (2.2), we can obtain a similar result for asymptotically tight bounds. Suppose we know that $f = \Theta(g)$ and that $g = \Theta(h)$. Then since $f = O(g)$ and $g = O(h)$, we know from part (a) that $f = O(h)$; since $f = \Omega(g)$ and $g = \Omega(h)$, we know from part (b) that $f = \Omega(h)$. It follows that $f = \Theta(h)$. Thus we have shown

(2.3) If $f = \Theta(g)$ and $g = \Theta(h)$, then $f = \Theta(h)$.

Sums of Functions It is also useful to have results that quantify the effect of adding two functions. First, if we have an asymptotic upper bound that applies to each of two functions f and g , then it applies to their sum.

(2.4) Suppose that f and g are two functions such that for some other function h , we have $f = O(h)$ and $g = O(h)$. Then $f + g = O(h)$.

Proof. We're given that for some constants c and n_0 , we have $f(n) \leq ch(n)$ for all $n \geq n_0$. Also, for some (potentially different) constants c' and n'_0 , we have $g(n) \leq c'h(n)$ for all $n \geq n'_0$. So consider any number n that is at least as large as both n_0 and n'_0 . We have $f(n) + g(n) \leq ch(n) + c'h(n)$. Thus $f(n) + g(n) \leq (c + c')h(n)$ for all $n \geq \max(n_0, n'_0)$, which is exactly what is required for showing that $f + g = O(h)$. ■

There is a generalization of this to sums of a fixed constant number of functions k , where k may be larger than two. The result can be stated precisely as follows; we omit the proof, since it is essentially the same as the proof of (2.4), adapted to sums consisting of k terms rather than just two.

(2.5) Let k be a fixed constant, and let $f_1, f_2, \dots, f_k$ and h be functions such that $f_i = O(h)$ for all i . Then $f_1 + f_2 + \dots + f_k = O(h)$.

There is also a consequence of (2.4) that covers the following kind of situation. It frequently happens that we're analyzing an algorithm with two high-level parts, and it is easy to show that one of the two parts is slower than the other. We'd like to be able to say that the running time of the whole algorithm is asymptotically comparable to the running time of the slow part. Since the overall running time is a sum of two functions (the running times of

the two parts), results on asymptotic bounds for sums of functions are directly relevant.

(2.6) *Suppose that f and g are two functions (taking nonnegative values) such that $g = O(f)$. Then $f + g = \Theta(f)$. In other words, f is an asymptotically tight bound for the combined function $f + g$.*

Proof. Clearly $f + g = \Omega(f)$, since for all $n \geq 0$, we have $f(n) + g(n) \geq f(n)$. So to complete the proof, we need to show that $f + g = O(f)$.

But this is a direct consequence of (2.4): we're given the fact that $g = O(f)$, and also $f = O(f)$ holds for any function, so by (2.4) we have $f + g = O(f)$. ■

This result also extends to the sum of any fixed, constant number of functions: the most rapidly growing among the functions is an asymptotically tight bound for the sum.

Asymptotic Bounds for Some Common Functions

There are a number of functions that come up repeatedly in the analysis of algorithms, and it is useful to consider the asymptotic properties of some of the most basic of these: polynomials, logarithms, and exponentials.

Polynomials Recall that a polynomial is a function that can be written in the form $f(n) = a_0 + a_1n + a_2n^2 + \cdots + a_dn^d$ for some integer constant $d > 0$, where the final coefficient a_d is nonzero. This value d is called the *degree* of the polynomial. For example, the functions of the form $pn^2 + qn + r$ (with $p \neq 0$) that we considered earlier are polynomials of degree 2.

A basic fact about polynomials is that their asymptotic rate of growth is determined by their “high-order term”—the one that determines the degree. We state this more formally in the following claim. Since we are concerned here only with functions that take nonnegative values, we will restrict our attention to polynomials for which the high-order term has a positive coefficient $a_d > 0$.

(2.7) *Let f be a polynomial of degree d , in which the coefficient a_d is positive. Then $f = O(n^d)$.*

Proof. We write $f = a_0 + a_1n + a_2n^2 + \cdots + a_dn^d$, where $a_d > 0$. The upper bound is a direct application of (2.5). First, notice that coefficients a_j for $j < d$ may be negative, but in any case we have $a_jn^j \leq |a_j|n^d$ for all $n \geq 1$. Thus each term in the polynomial is $O(n^d)$. Since f is a sum of a constant number of functions, each of which is $O(n^d)$, it follows from (2.5) that f is $O(n^d)$. ■

One can also show that under the conditions of (2.7), we have $f = \Omega(n^d)$, and hence it follows that in fact $f = \Theta(n^d)$.

This is a good point at which to discuss the relationship between these types of asymptotic bounds and the notion of *polynomial time*, which we arrived at in the previous section as a way to formalize the more elusive concept of efficiency. Using $O(\cdot)$ notation, it's easy to formally define polynomial time: a *polynomial-time algorithm* is one whose running time $T(n)$ is $O(n^d)$ for some constant d , where d is independent of the input size.

So algorithms with running-time bounds like $O(n^2)$ and $O(n^3)$ are polynomial-time algorithms. But it's important to realize that an algorithm can be polynomial time even if its running time is not written as n raised to some integer power. To begin with, a number of algorithms have running times of the form $O(n^x)$ for some number x that is not an integer. For example, in Chapter 5 we will see an algorithm whose running time is $O(n^{1.59})$; we will also see exponents less than 1, as in bounds like $O(\sqrt{n}) = O(n^{1/2})$.

To take another common kind of example, we will see many algorithms whose running times have the form $O(n \log n)$. Such algorithms are also polynomial time: as we will see next, $\log n \leq n$ for all $n \geq 1$, and hence $n \log n \leq n^2$ for all $n \geq 1$. In other words, if an algorithm has running time $O(n \log n)$, then it also has running time $O(n^2)$, and so it is a polynomial-time algorithm.

Logarithms Recall that $\log_b n$ is the number x such that $b^x = n$. One way to get an approximate sense of how fast $\log_b n$ grows is to note that, if we round it down to the nearest integer, it is one less than the number of digits in the base- b representation of the number n . (Thus, for example, $1 + \log_2 n$, rounded down, is the number of bits needed to represent n .)

So logarithms are very slowly growing functions. In particular, for every base b , the function $\log_b n$ is asymptotically bounded by every function of the form n^x , even for (noninteger) values of x arbitrary close to 0.

(2.8) For every $b > 1$ and every $x > 0$, we have $\log_b n = O(n^x)$.

One can directly translate between logarithms of different bases using the following fundamental identity:

$$\log_a n = \frac{\log_b n}{\log_b a}.$$

This equation explains why you'll often notice people writing bounds like $O(\log n)$ without indicating the base of the logarithm. This is not sloppy usage: the identity above says that $\log_a n = \frac{1}{\log_b a} \cdot \log_b n$, so the point is that $\log_a n = \Theta(\log_b n)$, and the base of the logarithm is not important when writing bounds using asymptotic notation.

Exponentials Exponential functions are functions of the form $f(n) = r^n$ for some constant base r . Here we will be concerned with the case in which $r > 1$, which results in a very fast-growing function.

In particular, where polynomials raise n to a fixed exponent, exponentials raise a fixed number to n as a power; this leads to much faster rates of growth. One way to summarize the relationship between polynomials and exponentials is as follows.

(2.9) For every $r > 1$ and every $d > 0$, we have $n^d = O(r^n)$.

In particular, every exponential grows faster than every polynomial. And as we saw in Table 2.1, when you plug in actual values of n , the differences in growth rates are really quite impressive.

Just as people write $O(\log n)$ without specifying the base, you'll also see people write "The running time of this algorithm is exponential," without specifying which exponential function they have in mind. Unlike the liberal use of $\log n$, which is justified by ignoring constant factors, this generic use of the term "exponential" is somewhat sloppy. In particular, for different bases $r > s > 1$, it is never the case that $r^n = \Theta(s^n)$. Indeed, this would require that for some constant $c > 0$, we would have $r^n \leq cs^n$ for all sufficiently large n . But rearranging this inequality would give $(r/s)^n \leq c$ for all sufficiently large n . Since $r > s$, the expression $(r/s)^n$ is tending to infinity with n , and so it cannot possibly remain bounded by a fixed constant c .

So asymptotically speaking, exponential functions are all different. Still, it's usually clear what people intend when they inexactly write "The running time of this algorithm is exponential"—they typically mean that the running time grows at least as fast as *some* exponential function, and all exponentials grow so fast that we can effectively dismiss this algorithm without working out further details of the exact running time. This is not entirely fair. Occasionally there's more going on with an exponential algorithm than first appears, as we'll see, for example, in Chapter 10; but as we argued in the first section of this chapter, it's a reasonable rule of thumb.

Taken together, then, logarithms, polynomials, and exponentials serve as useful landmarks in the range of possible functions that you encounter when analyzing running times. Logarithms grow more slowly than polynomials, and polynomials grow more slowly than exponentials.

2.3 Implementing the Stable Matching Algorithm Using Lists and Arrays

We've now seen a general approach for expressing bounds on the running time of an algorithm. In order to asymptotically analyze the running time of